Deliverable 3 - INF201

Link to GitHub repository: https://github.com/fridauboe/Deliverable_3

Task 1: Update the Character class

In [1]:
```
from game_simulator_v1 import Character
```

In [2]:
```
# Create two characters with different HP and attack damage
hero = Character("Hero", hp_max=100, damage=10)
enemy = Character("Enemy", hp_max=80, damage=15)

# Test the hero's attack against the enemy
hero.attack(enemy)
print("Enemy HP after attack:", enemy.hp)

# Test that the hero loses 30 HP
hero.take_damage(30)
print("Hero HP after damage:", hero.hp)

# Test that the hero regains 20 HP
hero.heal(20)
print("Hero HP after healing:", hero.hp)

# Check if the hero is still alive
print("Hero is alive:", hero.is_alive)

# Test that the enemy's HP does not go under 0
enemy.take_damage(100)
print("Enemy HP:", enemy.hp)
print("Enemy is alive:", enemy.is_alive)
```

```
Enemy HP after attack: 70
Hero HP after damage: 70
Hero HP after healing: 90
Hero is alive: True
Enemy HP: 0
Enemy is alive: False
```

In Task 1, we made some changes to the `Character` class to improve how health and damage work. We added `take_damage()` and `heal()` to make sure HP stays between 0 and the maximum health. We also changed `attack()` so it uses `take_damage()` instead of changing the enemy's HP directly. Finally, we added `is_alive` to check whether a character is still alive. The tests show that everything works as expected.

Task 2: Introducing randomness with a Dice class

In [3]:
```
from game_simulator_v1 import Dice
```

In [4]:
```
# Test Dice class
d6 = Dice()
d20 = Dice(sides=20)

# Test representation
print(d6)
print(d20)

# Test roll
print("D6 roll:", d6.roll())
print("D20 roll:", d20.roll())

# Test __call__
print("D20 call:", d20())
```

```
Dice(sides=6)
Dice(sides=20)
D6 roll: 1
D20 roll: 16
D20 call: 4
```

In [5]:
```
# Test invalid number of sides
try:
    invalid_dice = Dice(sides=1)
except ValueError as e:
    print("Error:", e)
```

```
Error: A dice must have at least 2 sides
```

In Task 2, we created a `Dice` class to add randomness to the game. The class allows us to create dice with different numbers of sides, with 6 as the default. The `roll()` method generates a random number, while `__repr__()` shows the dice as text and `__call__()` lets us roll it directly. We tested the class using 6-sided and 20-sided dice and checked that an error occurs if a dice has fewer than 2 sides.

Task 3: Create items

In [6]:
```
from game_simulator_v1 import Character, Dice
from items import Weapon
```

In [7]:
```
# Test Weapon class
sword = Weapon("Sword", Dice(6))

print("Weapon name:", sword.name)
print("Damage dice:", sword.damage_dice)
print("Weapon damage:", sword.roll_damage())
```

```
Weapon name: Sword
Damage dice: Dice(sides=6)
Weapon damage: 4
```

In [8]:
```
# Test Character with weapon
from game_simulator_v1 import Character
hero = Character("Hero", weapon=sword)
enemy = Character("Enemy")

hero.attack(enemy)
```

```
print("Enemy HP after weapon attack:", enemy.hp)

# Test Character without weapon
enemy.attack(hero)
print("Hero HP after normal attack:", hero.hp)
```

```
Enemy HP after weapon attack: 94
Hero HP after normal attack: 90
```

In Task 3, we created a `Weapon` class that uses the `Dice` class to calculate random damage. We also updated the `Character` class so characters can have a weapon. If a character has a weapon, the weapon's damage is used instead of the normal damage. We tested attacks both with and without weapons, and the results show that everything works as expected. The weapon damage changes depending on the dice roll.

Task 4: Character specialization

In [9]:
```
from game_simulator_v1 import Warrior

# Create a warrior with 100 HP and 5 armor
warrior = Warrior("Warrior")

# Test damage reduction
warrior.take_damage(20)
print("Warrior HP after 20 damage:", warrior.hp)

# Test damage lower than armor
warrior.take_damage(3)
print("Warrior HP after 3 damage:", warrior.hp)
```

```
Warrior HP after 20 damage: 85
Warrior HP after 3 damage: 85
```

In [10]:
```
from game_simulator_v1 import Mage, Character

mage = Mage("Wizard")
enemy = Character("Enemy")

print("Mana before attack:", mage.mana)

mage.attack(enemy)

print("Mana after attack:", mage.mana)
print("Enemy HP after attack:", enemy.hp)
```

```
Mana before attack: 30
Mana after attack: 20
Enemy HP after attack: 80
```

In Task 4, we extended the `Character` class by creating two subclasses, `Warrior` and `Mage`. The `Warrior` has armor that reduces incoming damage, while the `Mage` uses mana to deal double damage with magic attacks. When the `Mage` has less than 10 mana, it uses a normal attack and restores 5 mana. By using inheritance and `super()`, both classes can use methods from `Character` while

having their own abilities. The tests show that the different abilities work as expected.

Task 5: Running battles

In [11]:
```
from game_simulator_v1 import Battle, Warrior, Mage

hero = Warrior("Hero")
enemy = Mage("Enemy")

battle = Battle(hero, enemy)

battle.simulate_battle()

print("Hero HP:", hero.hp)
print("Enemy HP:", enemy.hp)
print("Hero is alive:", hero.is_alive)
print("Enemy is alive:", enemy.is_alive)
```

```
Hero HP: 5
Enemy HP: 0
Hero is alive: True
Enemy is alive: False
```

In Task 5, we created a `Battle` class to simulate a fight between two characters. The `play_round()` method handles one round, where the hero attacks first and the enemy attacks back if it is still alive. The `simulate_battle()` method uses a `while` loop to repeat the rounds until one of the characters runs out of HP. The test shows that the battle works as expected and ends when one character is defeated.